



School of IT and Engineering  
CSCI3613 Artificial Intelligence  
Spring 2026

# Nardy AI

Team 7

Lala Huseynova BSCS

İlknur Huseynova BSCS

İbrahim Najafguliyev BSCS

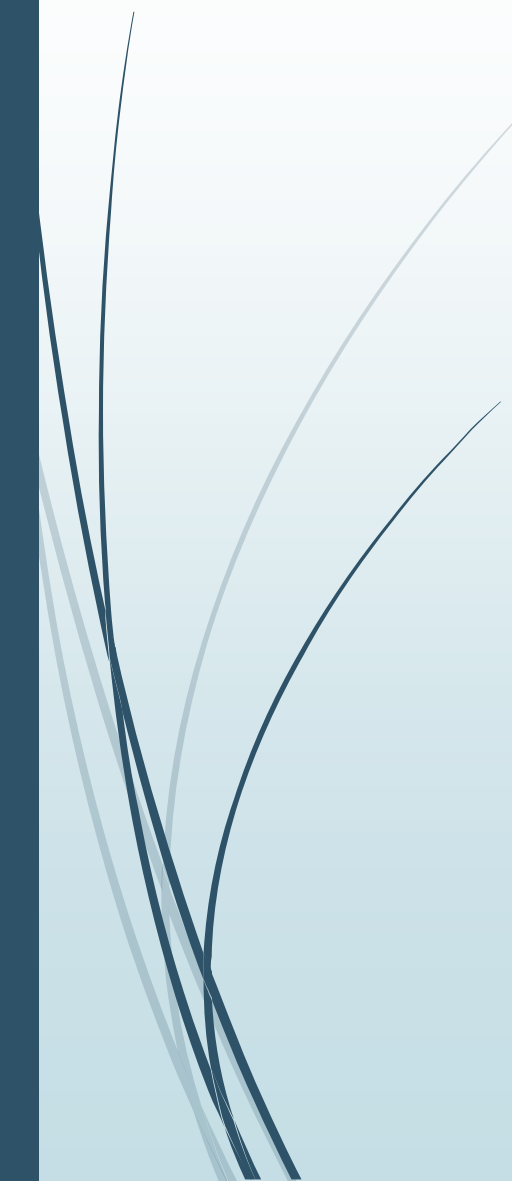
**AI opponent for a  
dice-based board  
game**



Expectiminimax + heuristic evaluation



# Agenda

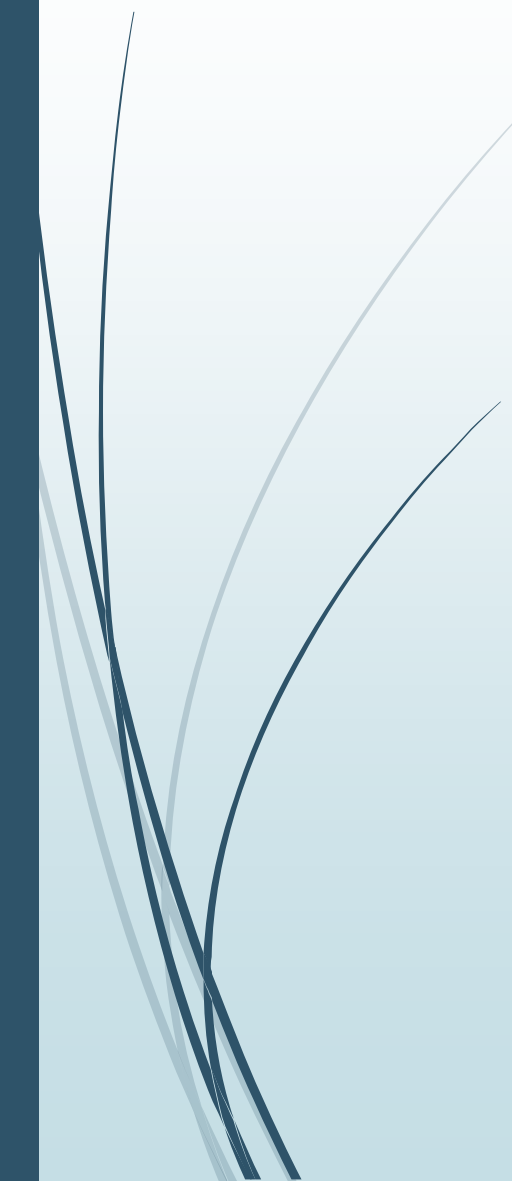


- Problem Statement
- Data
- Algorithm(s)
- Results
- Considerations

End goal: explain why the chosen AI is reasonable, how it was tested, and what still needs improvement.



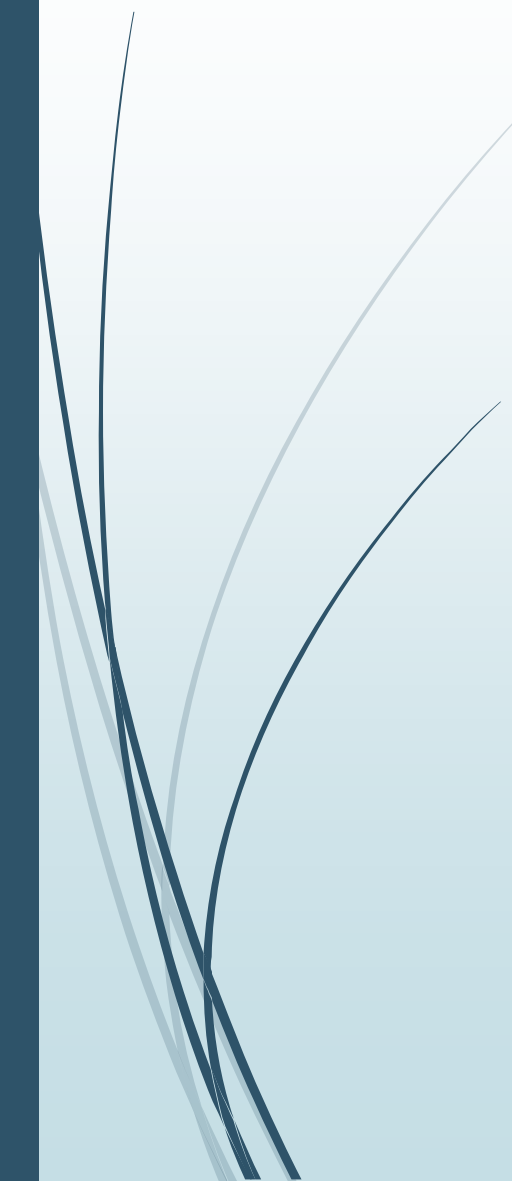
# Project in a Slide



|                     |                                                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Project Title       | Nardy AI                                                                                                         |
| Value               | Play Nardy against a computer that chooses moves using search instead of random selection.                       |
| Problem type        | Two-player stochastic game: deterministic legal moves + uncertain dice outcomes.                                 |
| Preprocessing steps | Encode board state, enumerate legal move sequences, model 21 dice outcomes, enforce head rule.                   |
| Algorithm(s)        | Expectiminimax search; weighted 5-feature heuristic; quick move ordering.                                        |
| Evaluation          | Rule correctness checks, AI-vs-Random comparison harness, decision-time observation, manual playtesting.         |
| Interpretation      | The agent is stronger than random choice in reasoning quality, but limited by search depth and heuristic design. |



# Contributions



| Team Member          | Contribution                                |
|----------------------|---------------------------------------------|
| Lala Huseynova       | Wrote the AI agent part (coding) and Report |
| İlknur Huseynova     | Created presentation and debugging          |
| İbrahim Najafguliyev | Wrote the report and coding                 |

# Problem Statement



- Nardy is a two-player board game where each player moves 15 checkers around a 24-point board and tries to bear them all off first.
- **Challenge:** Dice rolls introduce randomness every turn — the agent must pick the best move without knowing future rolls, while assuming the opponent always plays optimally.
- **Key Nardy rule:** Only one checker may leave the starting head point per turn (head rule) — checkers travel the full 24-point circuit before bearing off

# Data

- Nardy doesn't have any external dataset - states and actions come from game rules
- Board is stored as an array of 24 integers. Positive values represent player 1's checkers, negative values represent player 2's checkers.
- Only one checker may leave the head point per turn — tested separately before connecting to the AI

**21**

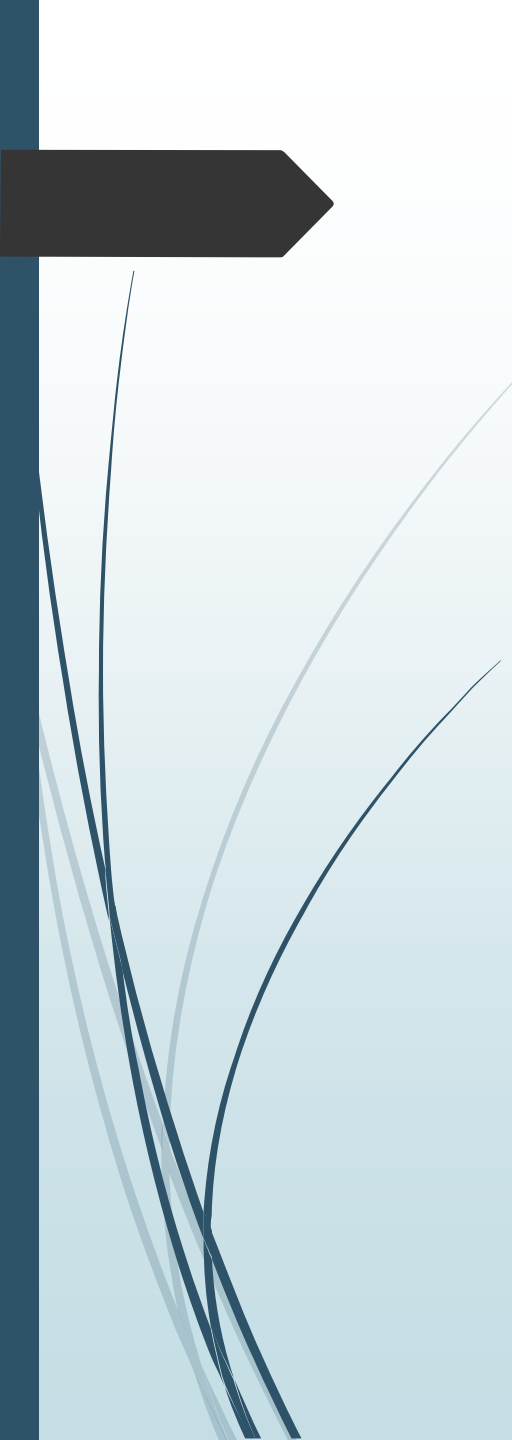
distinct dice  
outcomes

**6**

doubles, each  $1/36$

**15**

non-doubles, each  
 $2/36$



# Algorithm(s)

## ► Expectiminimax (depth 3)

Extends Minimax with chance nodes — instead of max/min, each dice outcome is weighted by its probability and averaged

## ► Heuristic evaluation (5 factors we score the board on):

Race progress (x1.2) — who is closer to finishing

Checkers borne off (x20) — who has removed more pieces

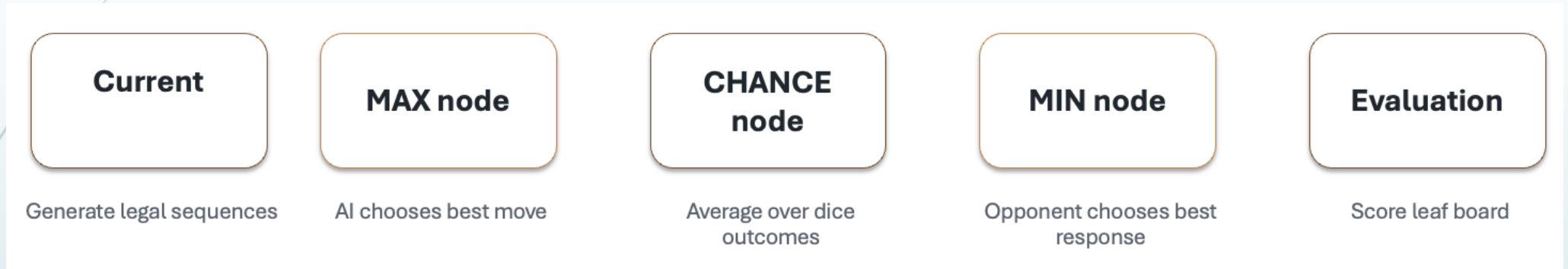
Exposed pieces (x8) — penalty for leaving lone checkers that can be blocked

Controlled points (x4) — reward for stacking 2+ checkers on a point

Home stretch (x3) — reward for having checkers near the finish

► **Move ordering:** Legal sequences are sorted by a quick score before full search. Better candidates are explored first, improving practical decision quality

# Algorithm(s)



**Why not plan minmax?** - Plain minimax assumes deterministic future turns.

Nardy has dice uncertainty, so chance nodes are required.

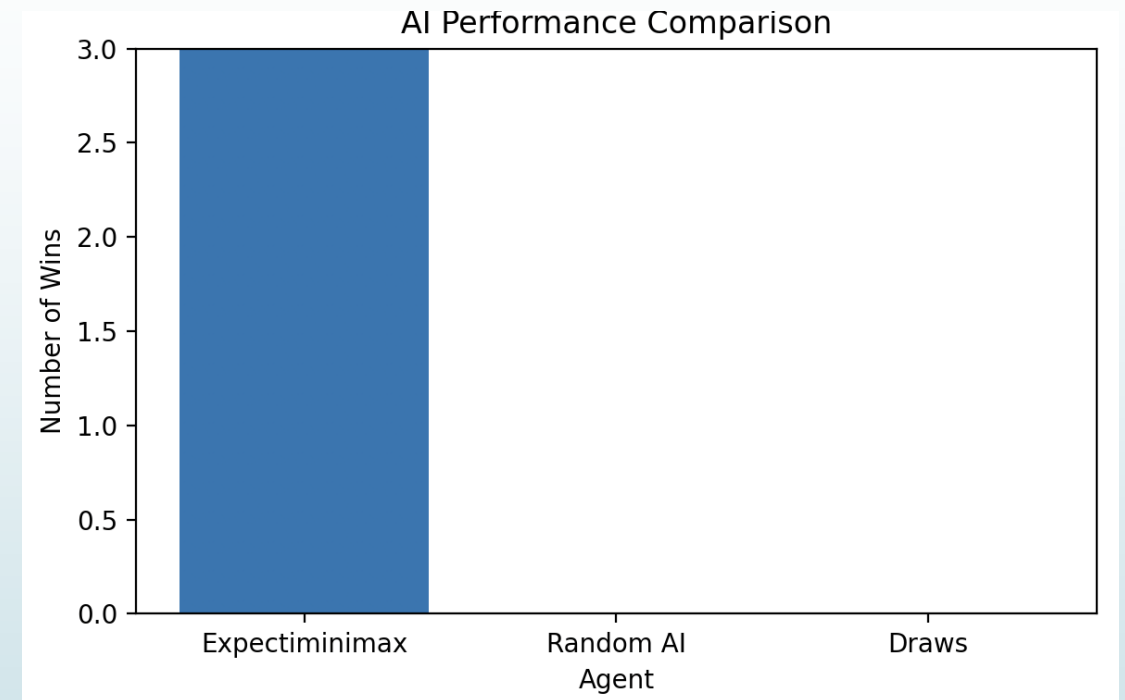
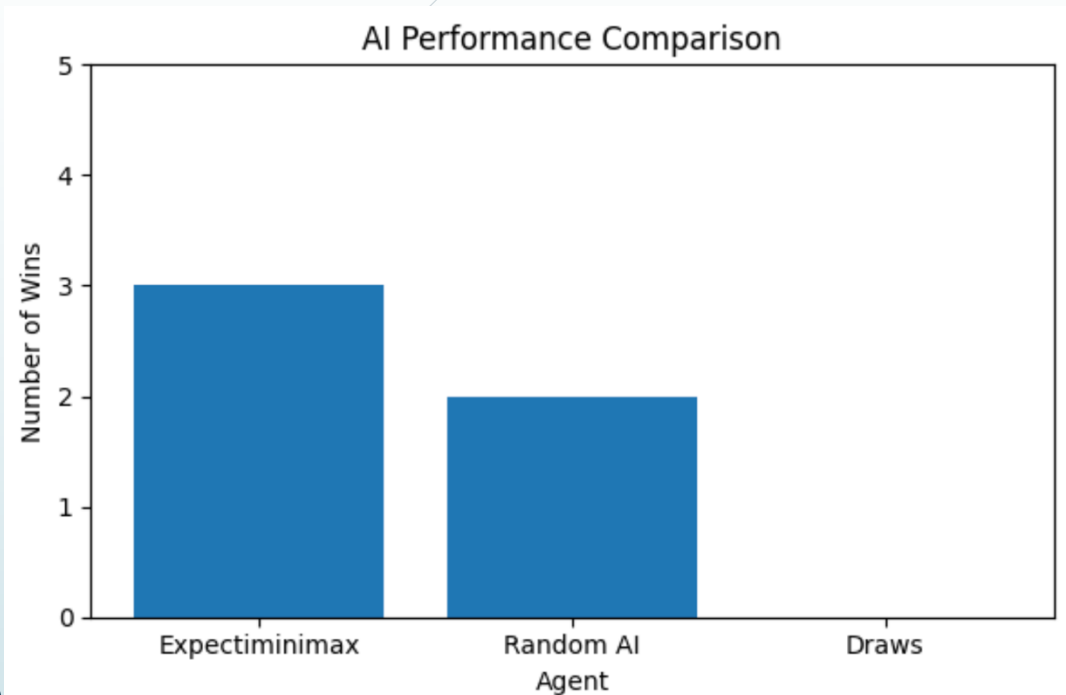
Expected value =  $\text{sum}(\text{probability} \times \text{child score})$ .

**Depth Used** - Agent default: depth 3 in NardyAI.

Comparison script may use depth 2 to keep runtime manageable.

Deeper search improves lookahead but increases branching quickly

# Results



- In the first test, Expectiminimax won 3 out of 3 games, while Random AI won 0 games.
- In the second test, Expectiminimax won 3 games, Random AI won 2 games, and there were 0 draws.

This shows that **Expectiminimax performed better overall**, but the Random AI could still win some games because of dice randomness.



# Considerations

## Limitations

There are many possible moves for each dice roll, so the AI can be slow.

Depth 3 helps the AI look ahead, but it still cannot see the whole game.

The heuristic values were chosen manually, not learned automatically.

## Future work

Add pruning methods where possible to reduce search time.

Cache repeated board states using a transposition table.

Tune heuristic weights using self-play or reinforcement learning.

Run larger experiments to report more reliable win rates.

**Main conclusion: this is a search-based Nardy agent, not a learned grandmaster bot — its strength comes from structured lookahead and a meaningful board evaluation.**



# THANK YOU FOR YOUR ATTENTION!

Questions?